

1. Arquitectura General del Sistema

El programa fue hecho con una **Programación Orientada a Objetos (POO)** donde fue utilizado el lenguaje de C# en formato Windows Forms .NET. Este funciona bajo controladores que el usuario activara mediante clicks sobre los mismos en la interfaz interactiva.

1.1 Estructura de Clases e Interfaz (POO)

Todo el juego el núcleo se concentra en la clase parcial Form 1: Form, en la cual se gestiona de manera uniforme todo el estado lógica de memoria y el renderizado grafico del tablero

- **Tablero** Matriz denominada char[,] de 8X8 que almacena de manera interna el estado de cada casilla. Los valores posibles son '\0' (vacía), 'B' (Ficha Negra / Jugador 1) y 'W' (Ficha Blanca / Jugador 2).
- **Botones:** Estructura de tipo Button[,] de 8X8 correlativa con la matriz lógica, encargada de la interacción visual con el usuario.
- **Vectores de Desplazamiento (dx, dy):** Arreglos de enteros que definen las 8 direcciones posibles (horizontal, vertical y diagonales) utilizados para recorrer las casillas de la matriz.

2. Lógica del Juego y Control de Matrices Gráficas

2.1 Inicialización Dinámica y Representación del Tablero

Al instanciarse la ventana de juego, se ejecutan de manera automática los dos métodos constructores de inicio:

- **CrearTablero():** Genera dinámicamente en tiempo de ejecución la cuadrícula de botones de tamaño 60X60 píxeles, estableciendo su color de fondo por defecto en verde (Color.ForestGreen). Cada botón se vincula con un evento de clic compartido (ClickCasilla()) y se le asigna su coordenada exacta utilizando la propiedad Tag mediante un objeto de tipo Point(i, j).
- **InicializarJuego():** Establece la configuración de apertura central clásica de Othello en el centro de la matriz:
 - Fichas Blancas ('W') en las posiciones [3, 3] y [4, 4].
 - Fichas Negras ('B') en las posiciones [3, 4] y [4, 3].

2.2 Ciclo Basado en Eventos (Game Loop Gráfico)

A diferencia de un programación en consola, este tipo de presentación nos permite ser más didácticos al no tener un “movimiento” lineal de escribir la acción si no un proceso como el siguiente:

[Usuario hace Clic] -> [ClickCasilla()] -> [Validar Vacío] -> [Pintar Ficha] -> [VoltearFichas()] -> [VerificarGanador()] -> [Cambio de Turno]

3. Matriz de Validaciones y Reglas de Movimiento (Othello)

El sistema valida las acciones del jugador de forma inmediata tras el disparo del evento clic.

Entidad / Situación	Regla de Validación Técnica	Acción si Falla / Comportamiento
Límites del Tablero	Validado automáticamente por el ciclo while mediante las cotas indexadas $x \geq 0 \ \&\& \ x < 8$ $\&\& \ y \geq 0 \ \&\& \ y < 8$.	El bucle interrumpe la búsqueda en esa dirección para evitar excepciones.
Casilla Ocupada	Evaluación directa sobre la matriz lógica: <code>if (tablero[fila, columna] != '\0')</code> .	Se ejecuta un return inmediato, ignorando la acción sin alterar el turno.
Mecánica de Captura / Volteo	A partir de la casilla jugada, se recorren las 8 direcciones posibles buscando secuencias de fichas enemigas que finalicen en una ficha del jugador actual.	Si se cumple la condición, todas las fichas rivales dentro de la línea de desborde se reasignan matemáticamente al color del jugador actual en la matriz y en la propiedad BackColor.

4. Sistema de Ataques y Condiciones de Fin de Partida

4.1 Sistema de Ataque (Captura por Encierro)

El sistema de “ataque” de este juego no consiste en tocas, pegar, comer, etc. A la pieza del rival, lo que haremos aquí será encerrarlas con nuestras piezas a las del rival. A la hora de colocar las fichas siempre llamaremos al método VoltearFichas():

1. Identifica el color del jugador enemigo ('W' o 'B').
2. Recorre las casillas continuas en las direcciones de los vectores dx y dy.
3. Al detectar que la línea de fichas enemigas está flanqueada por una ficha del jugador actual, ejecuta un ciclo interno while que sobrescribe la matriz tablero y cambia el BackColor de los botones intermedios al color del atacante.

4.2 Condición de Victoria y Cierre del Programa

El juego evalúa si ha finalizado en cada turno mediante el método VerificarGanador(), el cual cuenta cada casilla del tablero determinando a que jugador corresponde cada una:

- **Condición de activación:** Que la variable contadora “vacías” sea exactamente igual a 0 (tablero completamente lleno).
- **Determinación del Ganador:** Se comparan las variables acumuladoras “negras” y “blancas”. El sistema genera dinámicamente un string con el nombre del ganador (jugador1 o jugador2) o declara un "Empate" si los valores coinciden.
- **Cierre:** Se despliega un cuadro de diálogo MessageBox.Show() mostrando los puntajes totales acumulados y finaliza la ejecución llamando a Application.Exit().

(posible eliminación)

5. Módulo de Gestión de Puntajes (High Score)

El calculo de puntos se jala directamente del tablero al finalizar dependiendo de la cantidad de fichas del jugador para sumarlas a un acumulador.

5.1 Cálculo y Registro de Puntajes

- Cada ficha en el tablero representa el control de puntaje del jugador.
- El sistema extrae el valor final sumando las piezas finales de la sesión (negras o blancas).
- Este valor se procesa contra la lógica de persistencia en memoria del menú principal: si el nuevo puntaje calculado del ganador supera al récord previamente registrado en la sesión, el registro se actualiza con su nombre y puntuación.
- Si se consulta desde el menú principal y la variable global no registra partidas previas, se muestra en pantalla el literal estricto: "Aún no hay puntajes registrados".

6. Sistema de Login jugadores.

6.1 Funcionamiento lógico.

- Se creará un método denominado LoginJugadores(), en el cual llamaremos la función para crear un InputBox() el cual nos ayudará a ingresar el nombre de cada jugador.

- Al finalizar el input de este mismo tenemos una decisión lógica en la cual si la variable no está llena con ningún dato, se pondrá un dato predeterminado denominado “jugador 1” para el primer participante y “jugador 2” para el segundo usuario.